SMARTFOOD

Modules Restaurant & Réservation

Dossier technique : réalisations, architecture, règles métier et pistes d'intégration de l'intelligence artificielle

Flutter · Dart 3 | SQLite (sqflite) | QR code & PIN | API IA

AUTEUR
Omar Aguil

PROJET
SmartFood — 5SAE2 Mobile

BRANCHE
setup/structure-initiale

DATE
Novembre 2026

Sommaire

1 **Vue d'ensemble**
Les deux espaces, parcours client et parcours restaurant

2 **Architecture commune**
Couches, navigation, rafraîchissement automatique, dépendances

3 **Base de données**
Schéma v5, table des réservations, données de démonstration

4 **Module Réservation**
Cycle de vie, règles métier, transaction, retrait QR / PIN, écrans

5 **Module Restaurant**
Catalogue client, dashboard, gestion des plats et du stock

6 **Profil client & tests**
Module utilisateur, tests automatisés, limites actuelles

7 **Intégration d'une API d'IA**
Architecture sécurisée, six idées, modèles et coûts, plan

1 Vue d'ensemble

Deux espaces dans une même application

L'application propose **deux espaces**. Il n'y a pas encore d'authentification : un bouton dans la barre de navigation permet de basculer de l'un à l'autre pour les tests.

Espace	Onglets de la barre du bas	Rôle
Client	Accueil · Réservations · Profil · *Espace resto*	Parcourir les restaurants, consulter les plats, réserver, suivre ses réservations
Restaurant	Accueil · Suivi · Plats · Historique · *Espace client*	Gérer **un seul restaurant de test** : plats, stock, réservations, retrait par QR / PIN

Parcours client

Liste des restaurants → Plats du restaurant → Détail du plat → Formulaire de réservation → Confirmation : QR + PIN → Mes réservations

Le bouton « Réserver » présent sur chaque carte de plat mène directement au formulaire, sans passer par le détail.

Parcours restaurant

Accueil (Dashboard)

Indicateurs du jour, liste « À confirmer », bouton pour appeler le client.

Suivi en temps réel

Sections À confirmer / En cuisine / Prêtes, avancement du statut en un geste.

Retrait client

Scan du QR ou saisie du PIN : seul moyen d'atteindre le statut final « Récupérée ».

Plats & Historique

Ajout, modification, stock –/+, disponibilité ; historique filtrable par statut.

2 Architecture commune

Organisation du code et mécanismes partagés

Organisation en couches

models → **repositories** → **services** → **screens / widgets**

- **Repository** : accès SQLite uniquement (requêtes, transactions).
- **Service** : toutes les **règles métier** (validation, transitions de statut, stock).
- **Screens** : affichage seulement, aucune logique métier.

Navigation — `lib/app/`

Fichier	Rôle
`role_scope.dart`	`enum AppRole { client, restaurant }` — bascule animée entre les deux espaces
`role_shell.dart`	Coquille générique : barre de navigation + pile de navigateurs imbriqués. La barre reste visible partout, re-taper un onglet revient à sa racine, gestion du bouton retour et des badges.
`client_shell.dart`	Onglets client ; badge = nombre de réservations actives
`restaurant_shell.dart`	Onglets restaurant ; badge = nombre de réservations « En attente »
`restaurant_tabs.dart`	Index des onglets restaurant (dashboard, monitor, dishes, history)

Rafraîchissement automatique

`lib/core/database/data_changes.dart` expose deux `ValueNotifier` (`catalog` et `reservations`). Chaque écriture en base les incrémente et les écrans abonnés se rechargent seuls : une réservation créée côté client apparaît immédiatement dans le Suivi du restaurant.

Utilitaire & dépendances

`lib/core/utils/phone_call.dart` : `callPhone(context, phone)` ouvre le composeur téléphonique, avec un message de repli si c'est impossible.

Package	Usage
`qr_flutter`	Affichage du QR code de la réservation
`mobile_scanner`	Scan du QR côté restaurant
`url_launcher`	Appeler le client

3 Base de données

SQLite — schéma en version 5

Migrations

Version	Ajout
1 – 2	`restaurants`, `menus` (module restaurant)
3	`reservations`
4	`profil_client`
5	`app_settings` (clé / valeur — stocke l'id du restaurant courant)

Table `reservations`

Colonne	Détail
`reference`	`SF-XXXXXX`, **unique**, générée avec un générateur aléatoire sécurisé
`menu_id`, `restaurant_id`	`ON DELETE SET NULL` : l'historique survit à la suppression d'un plat
`restaurant_nom`, `menu_nom`, `prix_unitaire`	**Copie figée** au moment de la réservation : le prix payé ne change pas si le plat change de prix
`quantite`	Strictement positive
`client_prenom`, `client_nom`, `telephone`, `note`	Informations du client
`heure_retrait`	Date et heure de retrait choisies
`statut`	Contrainte `CHECK` sur les 6 statuts
`code_pin`	4 chiffres, pour le retrait
`date_creation`	Horodatage

Données de démonstration

- **3 restaurants de démo** avec leurs plats, visibles côté client.
- **« Restaurant Test »** (Avenue Habib Bourguiba, Tunis) avec 5 plats, créé automatiquement : c'est le restaurant géré dans l'espace restaurant.

  Exemples : *Escalope panée* (stock 3, pour tester le stock bas) · *Jus d'orange* (stock 0, donc non disponible).

4 Module Réservation

`lib/features/reservations/`

4.1 Cycle de vie d'une réservation

EN_ATTENTE → CONFIRMEE → PREPARATION → PRETE → **RECUPEREE**

Depuis n'importe quel statut actif : → ANNULEE

Statut	Libellé	Qui peut y mettre la réservation
`EN_ATTENTE`	En attente	Création par le client
`CONFIRMEE`	Confirmée	Restaurant
`PREPARATION`	En préparation	Restaurant
`PRETE`	Prête	Restaurant
`RECUPEREE`	Récupérée	**Uniquement par scan du QR ou saisie du PIN**
`ANNULEE`	Annulée	Client (si En attente ou Confirmée) ou restaurant (refus / annulation)

> **Règle clé**
> Le restaurant peut passer librement d'un statut actif à un autre (avancer ou corriger), **sauf** vers le statut final « Récupérée », réservé à la validation du retrait.

4.2 Règles métier

`services/reservation_service.dart`

Règle	Valeur
Horaires de retrait	8h00 → 23h00
Délai minimum de préparation	20 minutes après la réservation
Réservation à l'avance	Aujourd'hui ou demain
Quantité maximale	10 par réservation, et jamais plus que le stock restant
Nom et prénom	Obligatoires, longueur validée
Téléphone	Indicatif pays ; pour la Tunisie (+216) : exactement 8 chiffres
Plat réservable	Restaurant actif **et** plat disponible **et** stock > 0

4.3 Création sécurisée par transaction

`repositories/reservation_repository.dart`

La création se fait dans **une seule transaction SQLite** :

1. Vérifier que le restaurant est actif.
2. Vérifier que le plat est disponible et que le stock suffit (relu en base, pas depuis l'écran).
3. Décrémenter le stock **et** passer le plat en « non disponible » s'il atteint 0, en une seule requête.
4. Insérer la réservation avec sa référence unique et son PIN.

```
UPDATE menus
SET quantite   = quantite - ?,
    disponible = CASE WHEN quantite - ? <= 0 THEN 0 ELSE disponible END
WHERE id = ?
```

> **Garantie**
> Si une étape échoue, rien n'est enregistré : pas de stock perdu, pas de réservation fantôme.

- **Annulation** : la quantité est remise en stock et le plat redevient disponible s'il était tombé à 0 (si le restaurant est actif).
- **Changement de statut** : verrou optimiste (`WHERE statut = ancien`). Si deux actions arrivent en même temps, la seconde est refusée au lieu d'écraser la première.

4.4 Code de retrait

PIN
4 chiffres, affiché au client après la réservation.

QR code
Contenu : `smartfood:REFERENCE:PIN`

La validation exige le statut `PRETE` et accepte le PIN, la référence ou le contenu du QR.

4.5 Écrans côté client

Écran	Contenu
Détail du plat	Nom, catégorie, description, prix, **quantité restante** (alerte « Plus que N portions »), restaurant, horaires, bouton « Réserver ce plat » désactivé si indisponible
Formulaire de réservation	Quantité, créneau de retrait, coordonnées pré-remplies depuis le profil (profil enregistré automatiquement à la première réservation), note, total
Confirmation	QR code, PIN, référence, récapitulatif, montant à payer sur place
Mes réservations	Liste avec les réservations actives en premier, mise à jour automatique
Détail d'une réservation	Suivi du statut, QR, annulation par glissement (si autorisée)

4.6 Écrans côté restaurant

Écran	Contenu
Suivi	Compteurs, alertes, sections À confirmer / En cuisine / Prêtes ; bouton « étape suivante » sur chaque carte ; bouton **Appeler** pour les réservations en attente ; rafraîchissement toutes les 30 s ; notification à l'arrivée d'une nouvelle réservation (action « Confirmer ») ; bouton flottant **Retrait client**
Détail d'une réservation	Informations complètes, changement de statut, Refuser / Annuler, carte « Réservation à confirmer » avec **Appeler** + **Confirmer**, validation par PIN / QR
Historique	Toutes les réservations, filtrables par statut
Retrait client	Scanner le QR **ou** taper le PIN (recherche automatique dès 4 chiffres) ; affiche le client, le plat et **le montant à encaisser**, puis valide

5 Module Restaurant

`lib/features/restaurants/` — créé par le binôme, repris et branché sur la base de données

5.1 Côté client

Écran	Contenu
Catalogue des restaurants	Liste des restaurants actifs, lue depuis SQLite
Plats du restaurant	Chaque carte a un bouton **Réserver** ; un appui sur la carte ouvre le détail

L'ancienne barre de navigation en double (Accueil / Restaurants / Favoris / Profil) a été supprimée au profit d'une barre unique.

5.2 Côté restaurant

Écran	Contenu
Accueil (Dashboard)	Encaissé aujourd'hui, réservations du jour, panier moyen, annulations sur 7 jours ; liste **« À confirmer »** avec Appeler / Confirmer ; graphique des 7 derniers jours ; top des plats ; état des plats (en vente, stock bas, épuisés)
Mes plats	Filtres ; stock **–/+** directement sur la carte ; interrupteur de disponibilité ; modifier / supprimer ; option « Remettre en vente »
Formulaire de plat	Ajout et modification avec validation
Mon restaurant	Statistiques, alertes cliquables, Fermer / Rouvrir, Modifier
Formulaire restaurant	Nom, adresse, ville, téléphone

5.3 Règles métier

MenuService

- Stock bas : 5 portions ou moins.
- Stock à 0 : plat **automatiquement non disponible**, impossible de le remettre disponible tant que le stock est vide.
- Ajouter du stock à un plat épuisé le remet en vente.
- Suppression **interdite** s'il reste des réservations actives.

RestaurantService

- Fermeture (archivage) **interdite** s'il reste des réservations actives.
- Réouverture possible.
- `CurrentRestaurant.id()` retrouve le restaurant de test enregistré dans `app_settings`.

6 Profil client & tests

Module utilisateur, qualité et limites

6.1 Profil client — `lib/features/users/`

- Profil (prénom, nom, indicatif pays, téléphone) stocké sur une seule ligne.
- Écran de modification du profil.
- Le profil pré-remplit automatiquement le formulaire de réservation.

6.2 Tests automatisés

13 tests — tous au vert
Commande : `flutter test`

Fichier	Couverture
`reservation_rules_test.dart`	12 tests des règles : horaires, délai de préparation, quantité maximale, validation du téléphone et du nom, transitions de statut
`widget_test.dart`	Navigation et bascule client ↔ restaurant

6.3 Limites actuelles

Base locale par appareil
Chaque téléphone a sa propre base SQLite : une réservation faite sur un téléphone n'existe pas sur un autre. Pour tester avec deux téléphones (client + restaurant), il faut un **backend partagé** (Firebase ou API REST).

- **Pas d'authentification** : la bascule client / restaurant est un bouton de test.
- **Un seul restaurant** géré côté restaurant (le restaurant de test).

7 Intégration d'une API d'IA

Propositions pour le module Réservation

7.1 Architecture — à respecter quelle que soit l'idée

Ne jamais mettre la clé d'API dans l'application Flutter
Un APK se décompile facilement : n'importe qui pourrait récupérer la clé et l'utiliser à vos frais.

Application Flutter → HTTP → Serveur relais (Node.js, Python ou Firebase Functions) — détient la clé → API Claude

Bonus
Ce serveur peut devenir le **backend partagé** qui manque aujourd'hui pour tester sur deux téléphones : un seul chantier règle les deux problèmes.

Bonnes pratiques

- N'envoyer **que les données utiles** : pas de numéro de téléphone ni de nom complet si l'IA n'en a pas besoin.
- Demander une **réponse en JSON structuré** pour que Flutter l'exploite sans analyser du texte libre.
- L'IA **propose**, l'application **valide** : toute suggestion repasse par les règles de `ReservationService` (stock, horaires, quantité).
- Prévoir un comportement de secours : l'application doit fonctionner normalement sans l'IA.

7.2 Six idées, de la plus simple à la plus ambitieuse

1 Assistant de réservation en langage naturel — Recommandée — Client

Le client écrit : « *Un plat pas trop cher pour 2 personnes, vers 13h, sans porc* ». L'IA reçoit la liste des plats disponibles et renvoie :

```
{ "menu_id": 12, "quantite": 2, "heure_retrait": "13:00",
  "explication": "Couscous poulet, 18 DT, 7 portions restantes" }
```

L'application ouvre le **formulaire de réservation pré-rempli** ; le client n'a plus qu'à confirmer. Le parcours existant est réutilisé tel quel.

2 Analyse intelligente des notes clients Restaurant

La note libre (« *sans oignons, allergie arachides, retard 10 min* ») devient des étiquettes :

```
{ "allergies": ["arachides"], "modifications": ["sans oignons"],
  "retard_minutes": 10, "priorite": "haute" }
```

Le restaurant voit des **badges colorés** (⚠ Allergie) sur les cartes du Suivi. Coût très faible, forte valeur pour la sécurité alimentaire.

3 Prévision de stock pour demain Dashboard

L'historique des 30 derniers jours (plat, quantité, jour, heure — sans données personnelles) est analysé ; l'IA renvoie une quantité conseillée par plat. Carte « Préparer pour demain » avec un bouton **Appliquer** qui ajuste le stock.

4 Brief du jour Dashboard

Un résumé en trois phrases : « *12 réservations aujourd'hui, pic entre 12h et 13h. 3 sont à confirmer. L'Escalope panée sera épuisée avant midi.* » L'IA rédige à partir des indicateurs déjà calculés.

5 Recommandations personnalisées Client

À partir de l'historique de « Mes réservations » : « *Vous aimez les plats tunisiens, essayez le Lablabi du Restaurant X* », en haut de l'accueil.

6 Message de confirmation préparé Restaurant

Pour une réservation « À confirmer », l'IA prépare un script d'appel ou un SMS, en français ou en arabe tunisien.

7.3 Choix du modèle et coûts (API Claude d'Anthropic)

Modèle	Identifiant	Entrée / sortie (par million de tokens)	Usage conseillé
Claude Opus 5.5	`claude-opus-5-5`	4 $ / 20 $	Choix par défaut : meilleure qualité (assistant, prévisions)
Claude Sonnet 5.5	`claude-sonnet-5-5`	2 $ / 10 $	Bon compromis si le budget est serré
Claude Haiku 5.5	`claude-haiku-5-5`	0,10 $ / 0,50 $	Tâches courtes et répétitives (étiquetage des notes)

Ordre de grandeur : analyser une note (environ 300 tokens en entrée, 100 en sortie) coûte une fraction de centime, même avec Opus. Pour un projet étudiant, le coût reste négligeable.

7.4 Exemple d'appel

Requête envoyée par le serveur relais

```
curl https://api.anthropic.com/v1/messages \
  -H "Content-Type: application/json" \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2026-10-21" \
  -d '{
    "model": "claude-opus-5-5",
    "max_tokens": 4096,
    "system": "Tu analyses les notes de commandes d un restaurant.",
    "messages": [
      {"role": "user", "content": "Note : sans oignons, allergie arachides, retard 10 min"}
    ]
  }'
```

Côté Flutter : appel du serveur relais uniquement, sans clé

```
// lib/features/reservations/services/ai_service.dart (proposition)
final response = await http.post(
  Uri.parse('$baseUrl/ai/analyser-note'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'note': reservation.note}),
);
```

7.5 Plan proposé

1. Monter le **serveur relais**, idéalement le futur backend partagé de l'équipe.
2. Implémenter **l'idée 2** (notes → badges) : la plus simple, pour valider la chaîne de bout en bout.
3. Implémenter **l'idée 1** (assistant de réservation) : l'effet le plus marquant en démonstration.
4. Selon le temps restant : **idée 3** (prévision de stock) ou **idée 4** (brief du jour).